R is the second language, not a new problem

CONCEPT & REASONING

Python has already taught us input, decisions, loops, functions and testing. R gives us another way to work with data. We will reuse a five-day expense problem.

APPLIED EXAMPLE / TAKEAWAY

QUESTION: What do the five amounts tell us about spending over a week?

Unit 7 learning goals

CONCEPT & REASONING

By the end, you should be able to run an R script, create vectors, summarise values, make decisions, build a data frame, draw a chart and verify expected results.

APPLIED EXAMPLE / TAKEAWAY

REQUIRED: base R only. RStudio is optional.

Why learn R?

CONCEPT & REASONING

R is designed for statistical computing, data manipulation and graphics. Python and R are complementary: choose tools based on the task, not fashion.

APPLIED EXAMPLE / TAKEAWAY

R: data -> check -> summarise -> visualise -> explain

R console and script

CONCEPT & REASONING

A console evaluates one command at a time. A script saves commands so you can reproduce the same result. RStudio can help edit files but is not a substitute for installing R.

R CODE / APPLIED EXAMPLE

```
2 + 3
print("Hello from R")
# Save other lines as first_steps.R
```

Locate a script

CONCEPT & REASONING

File paths matter when loading data. Use the current working directory to understand where R looks for a relative file name.

R CODE / APPLIED EXAMPLE

```
getwd()
list.files()
# source("first_steps.R")
```

The first labelled output

CONCEPT & REASONING

print() displays an object. cat() combines a label and values; use a newline so the next result starts on a new line.

R CODE / APPLIED EXAMPLE

```
fare <- 1500
trips <- 6
cat("Total in TZS:", fare * trips, "\n")
```

Assignment <-

CONCEPT & REASONING

R usually uses <- to assign a value to a name. == tests whether two values are equal; it does not store a value.

R CODE / APPLIED EXAMPLE

```
allowance <- 20000
allowance <- allowance + 5000
allowance == 25000  # TRUE
```

Variables and naming

CONCEPT & REASONING

Meaningful names such as daily_expenses and allowance make code easier to read. R is case-sensitive, so total and Total are different.

R CODE / APPLIED EXAMPLE

```
daily_expenses <- 10000
Daily_expenses <- 8000
print(daily_expenses) # 10000
```

Four useful data types

CONCEPT & REASONING

Double and integer values are numeric. Character values are text. Logical values are TRUE or FALSE. Check types before performing calculations.

R CODE / APPLIED EXAMPLE

```
typeof(5)       # double
typeof(5L)      # integer
typeof("TZS")   # character
typeof(TRUE)    # logical
```

What is an atomic vector?

CONCEPT & REASONING

A vector stores values of a common atomic type. The c() function combines elements; many R calculations operate across the vector at once.

R CODE / APPLIED EXAMPLE

```
expenses <- c(2000,3000,0,4000,1000)
length(expenses)      # 5
```

Mixing values changes type

CONCEPT & REASONING

A base-R atomic vector cannot keep unrelated element types as independent Python objects. Combining a number and text often converts all entries to character.

R CODE / APPLIED EXAMPLE

```
x <- c(500, "unknown")
typeof(x) # character
# Do not sum(x): it is not numeric.
```

R indexing starts at 1

CONCEPT & REASONING

The first element of a Python list has index zero. The first element of an R vector has index one. This is one of the most important translation differences.

R CODE / APPLIED EXAMPLE

```
expenses <- c(2000,3000,0,4000,1000)
expenses[1]   # 2000
expenses[5]   # 1000
```

1 : 2000	2 : 3000	3 : 0	4 : 4000	5 : 1000

Select several vector elements

CONCEPT & REASONING

Use c(positions) for particular elements and start:end for a continuous section. Index zero returns an empty result, not the first element.

R CODE / APPLIED EXAMPLE

```
expenses[c(1,3,5)] # 2000 0 1000
expenses[2:4]      # 3000 0 4000
expenses[0]        # numeric(0)
```

Create sequences

CONCEPT & REASONING

The colon creates a regular integer sequence. seq() creates sequences with a step, and rep() repeats values.

R CODE / APPLIED EXAMPLE

```
1:5                 # 1 2 3 4 5
seq(2, 10, by=2)    # 2 4 6 8 10
rep(0, times=5)     # 0 0 0 0 0
```

Arithmetic in R

CONCEPT & REASONING

Addition, subtraction, multiplication and division use familiar symbols. Unlike Python, exponentiation uses ^, and integer division uses %/%.

R CODE / APPLIED EXAMPLE

```
(1000 + 500) * 2   # 3000
2^3                # 8
7 %% 3             # 1
7 %/% 3            # 2
```

Vectorisation

CONCEPT & REASONING

Element-wise multiplication can replace a manual loop when two numeric vectors represent paired prices and quantities.

R CODE / APPLIED EXAMPLE

```
prices <- c(1000,1500,2000)
quantity <- c(2,1,3)
prices * quantity  # 2000 1500 6000
```

Avoid unintended recycling

CONCEPT & REASONING

R may repeat a shorter vector during element-wise operations. Check paired vectors have equal lengths before multiplying corresponding observations.

R CODE / APPLIED EXAMPLE

```
stopifnot(length(prices) == length(quantity))
sum(prices * quantity)  # 9500
```

Summarise five expenses

CONCEPT & REASONING

sum() and mean() answer different questions: total expenditure and average per day. length() counts entries, including zero observations.

R CODE / APPLIED EXAMPLE

```
expenses <- c(2000,3000,0,4000,1000)
sum(expenses)   # 10000
mean(expenses)  # 2000
length(expenses) # 5
```

Minimum, maximum and zero days

CONCEPT & REASONING

min() and max() describe the range of observations. A logical comparison counts zero-spend days because summing TRUE/FALSE acts like one/zero.

R CODE / APPLIED EXAMPLE

```
min(expenses)       # 0
max(expenses)       # 4000
sum(expenses == 0) # 1
```

Missing does not mean zero

CONCEPT & REASONING

An observed no-spend day is 0. An unknown amount is NA. These have different meanings and should not be silently mixed.

R CODE / APPLIED EXAMPLE

```
expenses <- c(2000, NA, 0, 4000, 1000)
sum(is.na(expenses)) # 1
mean(expenses)      # NA
```

Handle NA thoughtfully

CONCEPT & REASONING

The na.rm=TRUE option excludes missing entries. The denominator changes, so record this choice in your description of the average.

R CODE / APPLIED EXAMPLE

```
mean(c(2000,NA,0,4000,1000),
    na.rm=TRUE) # 1750
```

Comparisons produce logical values

CONCEPT & REASONING

For vectors, conditions produce TRUE/FALSE for each entry. Use these logical values to inspect which days exceeded a threshold.

R CODE / APPLIED EXAMPLE

```
expenses > 2000
# FALSE TRUE FALSE TRUE FALSE
```

Select values using a condition

CONCEPT & REASONING

A logical condition inside square brackets filters the original vector. This is useful for extracting unusually high fares or other observations.

R CODE / APPLIED EXAMPLE

```
expenses[expenses > 2000]
# 3000 4000
sum(expenses >= 3000) # 2
```

if / else in R

CONCEPT & REASONING

Use if with a single logical condition and braces to group statements. else if allows a third result. The decision logic from Python is unchanged.

R CODE / APPLIED EXAMPLE

```
if (total > allowance) {
  message <- "Over budget"
} else {
  message <- "Not over budget"
}
```

Handle the equality boundary

CONCEPT & REASONING

For budget classification, compare greater-than first, equality second and below otherwise. This avoids labelling equal expenditure as over budget.

R CODE / APPLIED EXAMPLE

```
if (total > allowance) {
  message <- "Over budget"
} else if (total == allowance) {
  message <- "Exactly on budget"
} else {
  message <- "Within budget"
}
```

ifelse() and a vector

CONCEPT & REASONING

Unlike a single if statement, ifelse() returns values chosen element by element for a condition vector.

R CODE / APPLIED EXAMPLE

```
ifelse(expenses == 0, "No spend", "Spend")
# "Spend" "Spend" "No spend" ...
```

for loops in R

CONCEPT & REASONING

Use for (value in values) to repeat statements over each element. It is useful to trace an algorithm, although sum() is simpler for a direct total.

R CODE / APPLIED EXAMPLE

```
total <- 0
for (value in expenses) {
  total <- total + value
}
print(total) # 10000
```

Loop safely through positions

CONCEPT & REASONING

seq_along(x) produces valid positions for a vector and avoids issues with an empty vector. Remember R indices begin at one.

R CODE / APPLIED EXAMPLE

```
for (i in seq_along(expenses)) {
  cat("Day", i, expenses[i], "\n")
}
```

Functions in R

CONCEPT & REASONING

A function accepts parameters and can calculate and return a value. Define reusable operations instead of duplicating code.

R CODE / APPLIED EXAMPLE

```
calculate_total <- function(x) {
  sum(x)
}
calculate_total(expenses) # 10000
```

Validate before a calculation

CONCEPT & REASONING

stopifnot() protects assumptions. For an expense report, require numeric finite non-negative values; if the condition is false, the script should stop.

R CODE / APPLIED EXAMPLE

```
stopifnot(is.numeric(expenses))
stopifnot(all(is.finite(expenses)))
stopifnot(all(expenses >= 0))
```

Data frames: rows and columns

CONCEPT & REASONING

A data frame stores different columns with names. Here, day labels are text and expense values are numeric; both columns have five rows.

R CODE / APPLIED EXAMPLE

```
report <- data.frame(
 day=c("Mon","Tue","Wed","Thu","Fri"),
 expense=c(2000,3000,0,4000,1000))
```

Inspect a data frame

CONCEPT & REASONING

head() previews rows, nrow() counts observations, and the $ operator selects a named column.

R CODE / APPLIED EXAMPLE

```
head(report)
nrow(report)          # 5
sum(report$expense)   # 10000
```

CSV import is optional

CONCEPT & REASONING

CSV files hold comma-separated text. A relative file name is resolved from R's working directory. Check the imported structure before calculating.

R CODE / APPLIED EXAMPLE

```
report <- read.csv("u7_expenses.csv")
str(report)
stopifnot(is.numeric(report$expense))
```

Export a report

CONCEPT & REASONING

write.csv() saves a table. Set row.names=FALSE when there is no need to export automatic row numbers.

R CODE / APPLIED EXAMPLE

```
write.csv(report, "my_report.csv",
          row.names=FALSE)
```

Your first R bar chart

CONCEPT & REASONING

barplot() in base R can visualise one numeric value per day. Include a title, the day labels and units on the vertical axis.

R CODE / APPLIED EXAMPLE

```
barplot(report$expense,
 names.arg=report$day,
 main="Five-day expenditure",
 ylab="Amount (TZS)", xlab="Day",
 col="steelblue")
```

Reading a chart responsibly

CONCEPT & REASONING

The tallest bar represents the highest observed expenditure. For the example, Thursday has TZS 4,000 and Wednesday has TZS 0. A graph cannot repair inaccurate data.

APPLIED EXAMPLE / TAKEAWAY

QUESTION: Which day has the highest cost?
ANSWER: Thursday, TZS 4,000.



Verify expected totals

CONCEPT & REASONING

Testing checks the script against requirements. Write expected totals independently, then assert that calculations match them.

R CODE / APPLIED EXAMPLE

```
stopifnot(length(expenses) == 5)
stopifnot(sum(expenses) == 10000)
stopifnot(mean(expenses) == 2000)
```

Test boundaries

CONCEPT & REASONING

All-zero expenditure, exact equality and above-budget totals are useful tests. A single successful ordinary input is not enough.

R CODE / APPLIED EXAMPLE

```
stopifnot(sum(rep(0,5)) == 0)
stopifnot(sum(c(1,2,3,4,5)) == 15)
```

Python and R: same variable idea

CONCEPT & REASONING

Python uses = to assign a name; the usual beginner R notation is <-. Both names can later refer to a new value.

APPLIED EXAMPLE / TAKEAWAY

PYTHON: allowance = 20000
R: allowance <- 20000

Python and R: list versus vector

CONCEPT & REASONING

Python uses a list with square brackets and index zero. R uses an atomic vector with c() and one-based indexing.

APPLIED EXAMPLE / TAKEAWAY

```
PYTHON: x = [2000,3000,0]
        x[0]  -> 2000
R:       x <- c(2000,3000,0)
        x[1]  -> 2000
```

Python and R: function definitions

CONCEPT & REASONING

Python uses def and indentation. R uses function() with braces. Both support readable reusable processing.

APPLIED EXAMPLE / TAKEAWAY

PYTHON: def double(x):
    return x * 2
R: double <- function(x) {x * 2}

Independent activity: expense report

CONCEPT & REASONING

Build a base-R script that creates a five-day data frame, calculates totals/average/zero days, classifies the allowance and produces a labelled bar chart.

APPLIED EXAMPLE / TAKEAWAY

Submit: StudentID_U7.R and test notes.

Activity U7 required tests

CONCEPT & REASONING

Demonstrate five cases: a normal week, all-zero values, exact budget, above budget, and a second exact-budget case. Record expected and actual outputs.

R CODE / APPLIED EXAMPLE

TEST: c(2000,3000,0,4000,1000)
ALLOWANCE: 20000
EXPECTED: total=10000, average=2000

Independent evidence and assistance

CONCEPT & REASONING

Explain your algorithm and demonstrate what happens when data changes. Keep the script reproducible and declare help from people, websites and AI tools.

APPLIED EXAMPLE / TAKEAWAY

CHECK: code + five tests + chart + explanation.

Quick reference

CONCEPT & REASONING

Remember: <- assigns; c() combines; indices start at 1; sum()/mean() summarise; data.frame() builds a table; barplot() graphs.

APPLIED EXAMPLE / TAKEAWAY

R does not run inside this website. Install R to execute your scripts.

What follows in Unit 8?

CONCEPT & REASONING

Unit 8 will bring Python and R together in a short evidence-based mini-project. The same problem definition, testing discipline and communication skills will carry forward.

APPLIED EXAMPLE / TAKEAWAY

YOUR NEXT STEP: Finish Activity U7, then review your checklist.